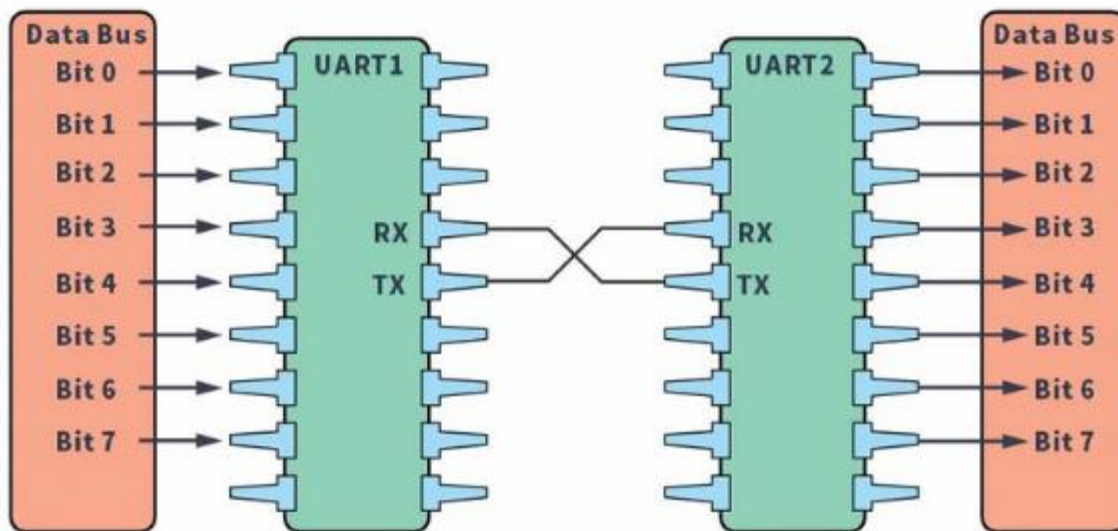


Full_Duplex_UART_Project



NTI_Summer training_final project

Digital design using FPGA

Prepared by:

Haneen Fady Kamel Shahin

UART Protocol:

UART (Universal Asynchronous Receiver-Transmitter) is an asynchronous serial communication protocol that enables data exchange between devices without requiring a shared clock. Data is transmitted sequentially over the serial line, with both communicating devices configured to use compatible communication parameters, including the data width and parity settings.

Each UART frame consists of a Start Bit, a configurable number of Data Bits, an optional Parity Bit, and a Stop Bit. The Start Bit indicates the beginning of a frame, data bits are transmitted Least Significant Bit (LSB) first, the optional Parity Bit provides basic error detection, and the Stop Bit marks the end of the frame. In this project, independent transmitter (TX) and receiver (RX) modules support full-duplex communication, allowing data transmission and reception to occur simultaneously, with configurable data width and even or odd parity modes.

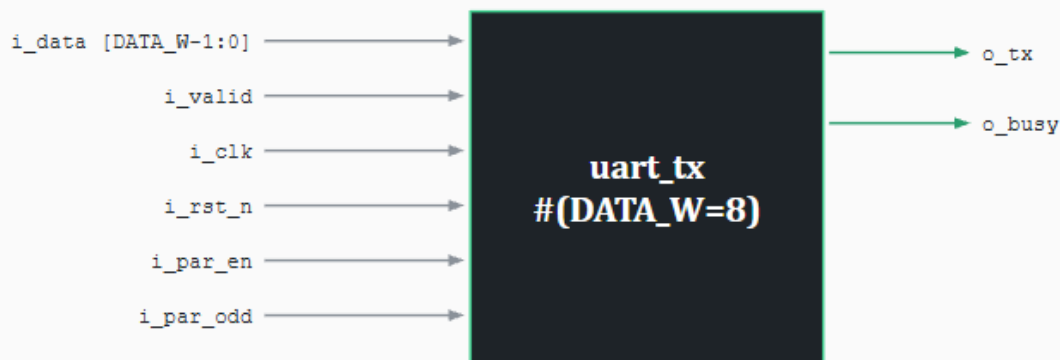
TX:

Overview:

1. RTL DATA FLOW & TOP-LEVEL ARCHITECTURE

The UART Transmitter (uart_tx) is a parameterizable digital IP designed to convert parallel input data (i_data) into an asynchronous serial bitstream (o_tx).

uart_tx — Port Diagram



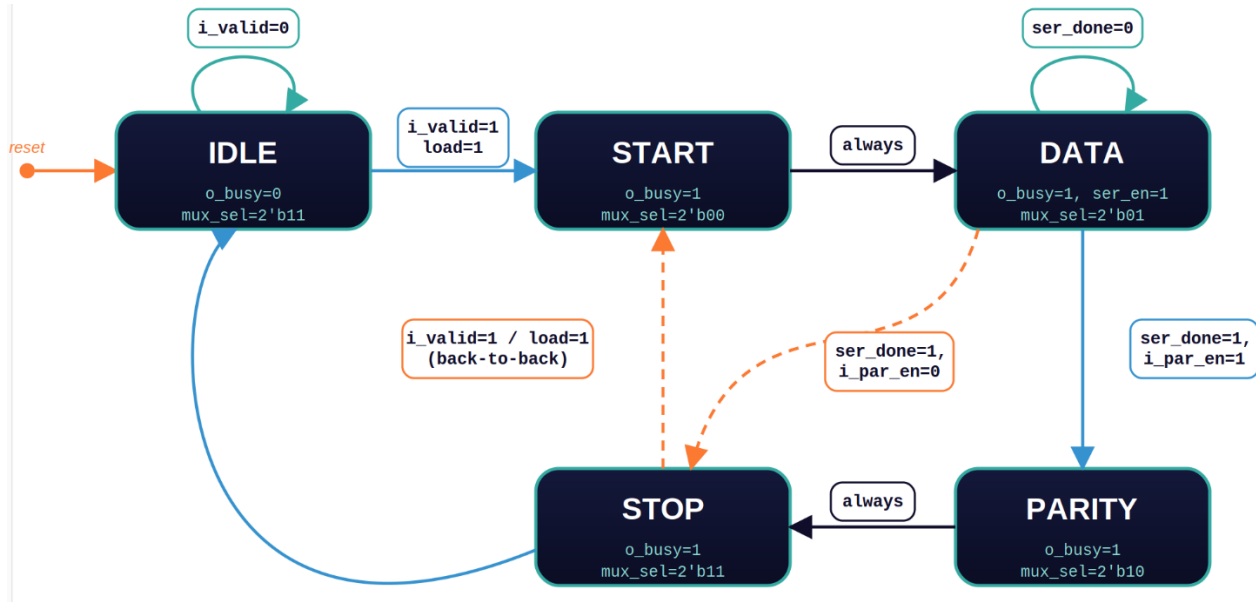
The hardware datapath and control flow proceed as follows:

A. IDLE State & Parallel Load:

- When the system is IDLE and a valid data flag ($i_valid = 1$) is asserted, the TX_FSM asserts the internal load signal.
- The Serializer module samples i_data into its internal shift register ($shift_reg \leftarrow i_data$) and resets its internal bit counter to zero ($counter \leftarrow 0$).
- The paritybit_calc module pre-calculates and registers the parity bit on the assertion of load.

B. Transmission Framing & State Sequence:

- **START State:** The FSM drives $mux_sel = 2'b00$, routing $start_bit (1'b0)$ to o_tx via the output multiplexer.
- **DATA State:** The FSM asserts $ser_en = 1$ and $mux_sel = 2'b01$.
 - The Serializer outputs $shift_reg[0]$ (LSB first) on o_serial_data , which is wired to $serial_bit$ at the top level.
 - On each clock cycle, $shift_reg$ shifts right by 1 bit, and counter increments until all DATA_W bits are shifted out.
 - Upon shifting the last bit, Serializer asserts the ser_done pulse.
- **PARITY State (Optional):** If parity is enabled ($i_par_en = 1$), the FSM transitions to PARITY ($mux_sel = 2'b10$).
 - The pre-calculated $parity_bit$ is driven to o_tx .
 - The paritybit_calc module outputs even parity (XOR) if $i_par_odd = 0$ or odd parity (XNOR) if $i_par_odd = 1$ over i_data .
- **STOP State:** The FSM drives $mux_sel = 2'b11$, routing $stop_bit (1'b1)$ to o_tx .
 - **Handshaking:** If i_valid is asserted high during the STOP state, load is asserted immediately to pre-load the next data payload for back-to-back framing.



2. SUBMODULE INTERCONNECTIVITY:

- TX_FSM: Controls state progression and generates control signals (load, ser_en, mux_sel, o_busy) based on inputs i_valid, i_par_en, and ser_done.
- Serializer: Parameterized payload shifter generating o_serial_data (connected to serial_bit at the top level) and ser_done.
- paritybit_calc: Combinational/registered parity generator outputting parity_bit.
- mux: Output routing multiplexer driving final o_tx.

3. VERIFICATION & TEST CASE STRATEGY:

To fully verify the uart_tx module, testbenches typically evaluate the following scenarios:

A. Reset Verification:

- Assert reset ($i_rst_n = 0$) to ensure FSM returns to IDLE, o_busy is deasserted (0), and shift counters are zeroed.

B. Parity Mode Verification (Even & Odd):

- Even Parity ($i_par_en = 1, i_par_odd = 0$): Verify parity bit equals XOR of i_data.
- Odd Parity ($i_par_en = 1, i_par_odd = 1$): Verify parity bit equals XNOR of i_data.

C. Standard Frame Transmissions (Parity Disabled):

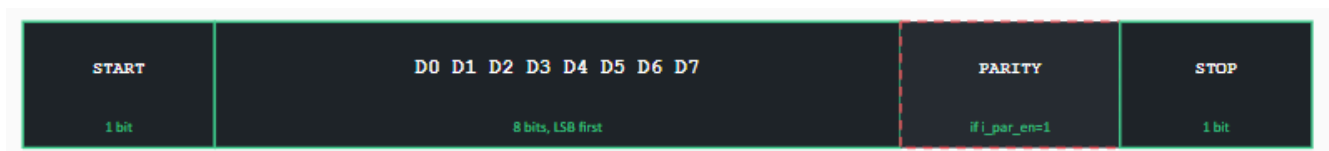
- Set `i_par_en = 0`, apply a valid payload (`i_data`) with `i_valid = 1`.
- Verify start bit (0), N data bits (LSB to MSB), and stop bit (1) timing.

D. Back-To-Back Transmissions:

- Assert `i_valid` while the current frame is in the STOP state.
- Verify that load asserts immediately, bypassing IDLE to transmit the next frame without line delay.

E. IDLE Line Stability:

- Ensure `o_tx` holds high (1'b1) continuously while in IDLE and `o_busy` remains 0.

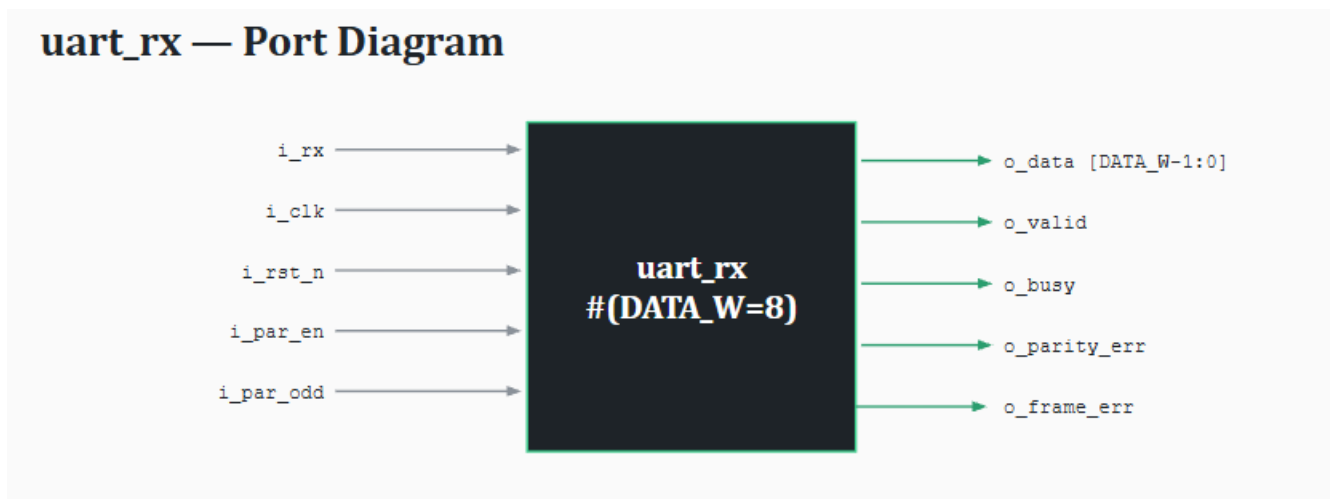


RX:

Overview:

1. RTL DATA FLOW & TOP-LEVEL ARCHITECTURE

The UART Receiver (`uart_rx`) is a parameterizable digital IP designed to convert an incoming asynchronous serial bitstream (`i_rx`) into a parallel output payload (`o_data`) along with status flags for frame validation.



The hardware datapath and control flow proceed as follows:

A. IDLE State & Start Bit Detection:

- When the system is `IDLE`, the receiver monitors the incoming serial line (`i_rx`).
- Upon detecting a falling edge (`i_rx = 0`), the `start_bit_detector` asserts `start_detected` and the `RX_FSM` transitions directly from `IDLE` to the `DATA` state.
- The falling edge is detected by comparing `i_rx` with its one-cycle-delayed copy (`i_rx_dly`):
$$\text{neg_edge_rx} = \sim i_rx \ \& \ i_rx_dly.$$

B. Reception Framing & State Sequence:

• Start Detection (IDLE to DATA):

When `start_detected` is asserted, the `RX_FSM` moves to the `DATA` state and asserts `shift_en` to enable the `de_serializer`.

- **DATA State:**

The FSM asserts `shift_en = 1`.

- The `de_serializer` samples incoming serial bits at mid-bit intervals and shifts them into an internal register (LSB first).
- On each bit sample, internal bit counting increments until all `DATA_W` bits are gathered.
- Upon capturing the last bit, the `de_serializer` asserts the `done` pulse.

- **PARITY State (Optional):**

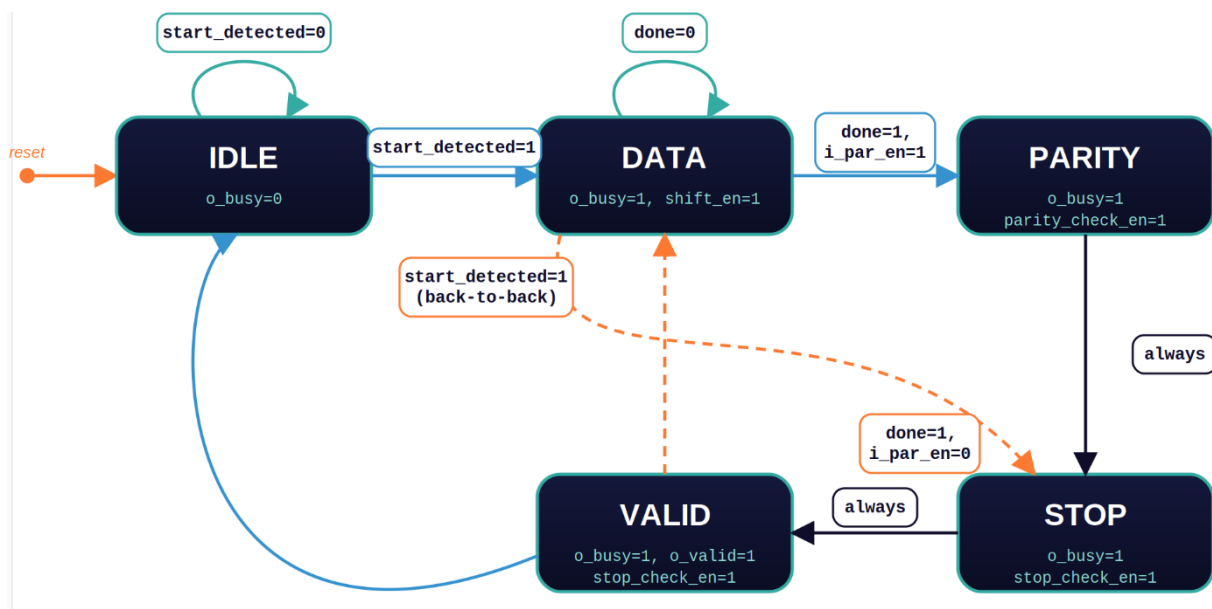
If parity is enabled (`i_par_en = 1`), the FSM transitions to `PARITY`.

- The incoming bit on `i_rx` is sampled and compared against the calculated parity of the received payload via `rx_checker`.
- If a mismatch occurs, the parity error flag (`o_parity_err`) is asserted.

- **STOP State:**

The FSM transitions to `STOP` to sample the stop bit (1'b1).

- **Framing Check:** If `i_rx` is sampled low during the `STOP` state, a framing error flag (`o_frame_err`) is asserted.
- **Output Latching:** Upon successful frame reception, the `RX_FSM` enters the `VALID` state and the parallel output flag (`o_valid`) pulses high for one clock cycle, updating `o_data`.



2. SUBMODULE INTERCONNECTIVITY:

- `RX_FSM`: Controls receiver state progression and generates enable signals based on line activity, frame parameters, and module completion flags.
- `de_serializer`: Parameterized serial-to-parallel converter that samples `i_rx` and shifts bits into the parallel payload register.
- `rx_checker`: Calculates expected parity across received data bits and checks for discrepancies against the sampled parity bit.
- `start_bit_detector`: Detects the falling edge on `i_rx` (`neg_edge_rx`) and generates `start_detected` to trigger the `RX_FSM`.

3. VERIFICATION & TEST CASE STRATEGY:

To fully verify the `uart_rx` module, testbenches typically evaluate the following scenarios:

A. Reset & Glitch Verification:

- Assert reset (`i_rst_n = 0`) to ensure FSM returns to `IDLE`, `o_valid` is low, and flags/counters are cleared.
- Inject short negative pulses (glitches) on `i_rx` in `IDLE` to verify glitch rejection without triggering false reception.

B. Parity Mode Verification (Even & Odd):

- Even Parity (`i_par_en = 1, i_par_odd = 0`): Verify correct reception when parity matches, and assertion of `o_parity_err` when a parity bit error is injected.
- Odd Parity (`i_par_en = 1, i_par_odd = 1`): Verify correct odd-parity checking and error detection.

C. Standard Frame Reception (Parity Disabled):

- Drive a valid UART frame on `i_rx` with `i_par_en = 0`. Verify `o_data` matches transmitted payload and `o_valid` pulses high.

D. Framing Error Detection:

- Drive a frame where the stop bit is corrupted (`i_rx = 0` during `STOP` state). Verify `o_frame_err` is asserted.

E. Back-To-Back Frame Reception:

- Drive sequential frames with zero inter-frame gap. Verify that the receiver continuously latches consecutive payloads without losing sync.

4. FUNCTIONAL SIMULATION & VERIFICATION RESULTS:

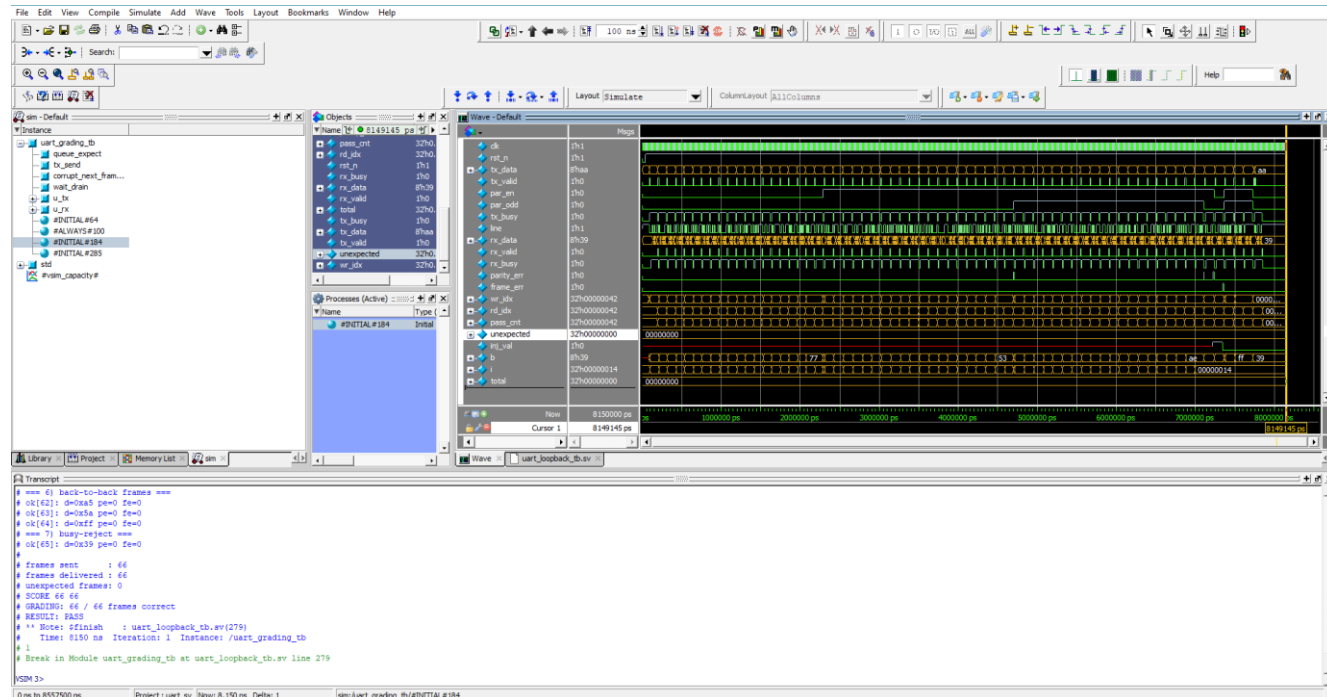
To validate the physical functionality and protocol compliance of the design, RTL behavioral simulation was performed using **Siemens QuestaSim**.

A. Testbench Execution & Waveform Analysis:

- **Signal Alignment:** As observed in the QuestaSim waveform window, all internal state transitions (cs/ns), handshake pulses (load, ser_en, ser_done), and output framing (o_tx / i_rx) strictly adhere to the expected timing specification.
- **Data Integrity:** Serialized bit sequences accurately match the input parallel payload (i_data / o_data), verifying correct LSB-first transmission/reception and parity bit calculation.

B. Simulation Transcript Summary:

- **Test Case Coverage:** The simulation transcript confirms successful execution across all programmed verification scenarios, including standard data frames, parity check modes (even/odd), and back-to-back transfer timing.
- **Pass/Fail Verdict:** The testbench self-checking logic completed with **0 Errors** and **0 Warnings**.



Linting of UART TX and RX IP Blocks:

RTL linting was performed using Questa Lint to assess the design against established RTL coding guidelines and identify potential coding issues, structural violations, and design-related warnings prior to synthesis.

The screenshot shows the Questa Lint 2021.1 interface. The left pane displays the Verilog code for the TX_FSM module. The right pane shows a schematic diagram of the module. The bottom pane shows the Lint Checks table with three warnings.

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Warning	Warning	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter ...	Serializer	Nomenclature Style	resolved	ME	1.1.4.3, 3.2.2.2
Warning	Warning	multi_ports_in_single_line		Multiple ports are declared in one line. Module Serializer, File D:/qfor...	Serializer	Rtl Design Style	resolved	ME	3.5.6.3
Warning	Warning	multi_ports_in_single_line		Multiple ports are declared in one line. Module TX_FSM, File D:/qfor...	TX_FSM	Rtl Design Style	resolved	ME	3.5.6.3

The screenshot shows the Questa Lint 2021.1 interface. The left pane displays the Verilog code for the RX_FSM module. The right pane shows a schematic diagram of the module. The bottom pane shows the Lint Checks table with three warnings.

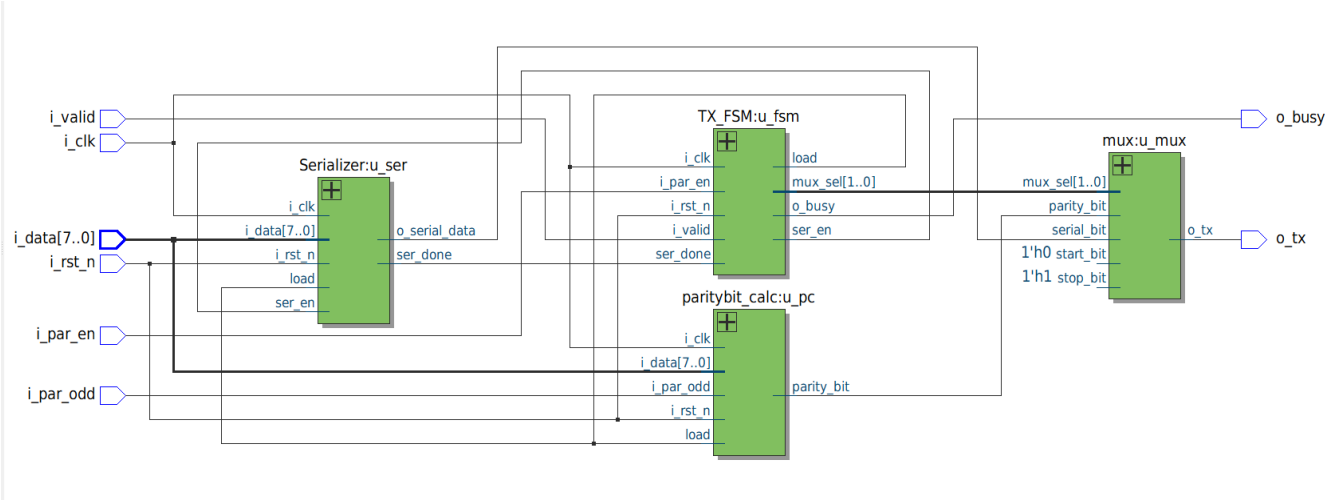
Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Warning	Warning	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter ...	RX_FSM	Nomenclature Style	resolved	ME	1.1.4.3, 3.2.2.2
Warning	Warning	always_signal_assignment_large		Always block has more signal assignments than the specified limit. T...	RX_FSM	Rtl Design Style	resolved	ME	2.6.1.3
Warning	Warning	multi_ports_in_single_line		Multiple ports are declared in one line. Module rx_checker, File D:/qfor...	rx_checker	Rtl Design Style	resolved	ME	3.5.6.3

Synthesis and Implementation of UART TX and RX IP Blocks:

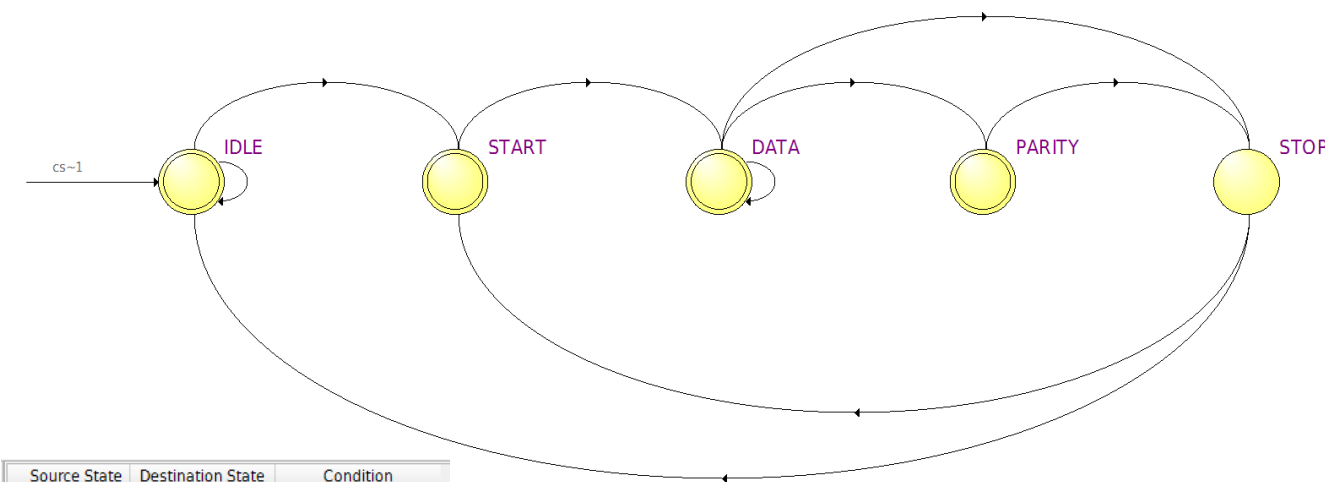
Quartus Prime was used to synthesize and implement the verified RTL, validating its successful mapping, placement, and routing for FPGA realization.

UART TX:

RTL Architecture:



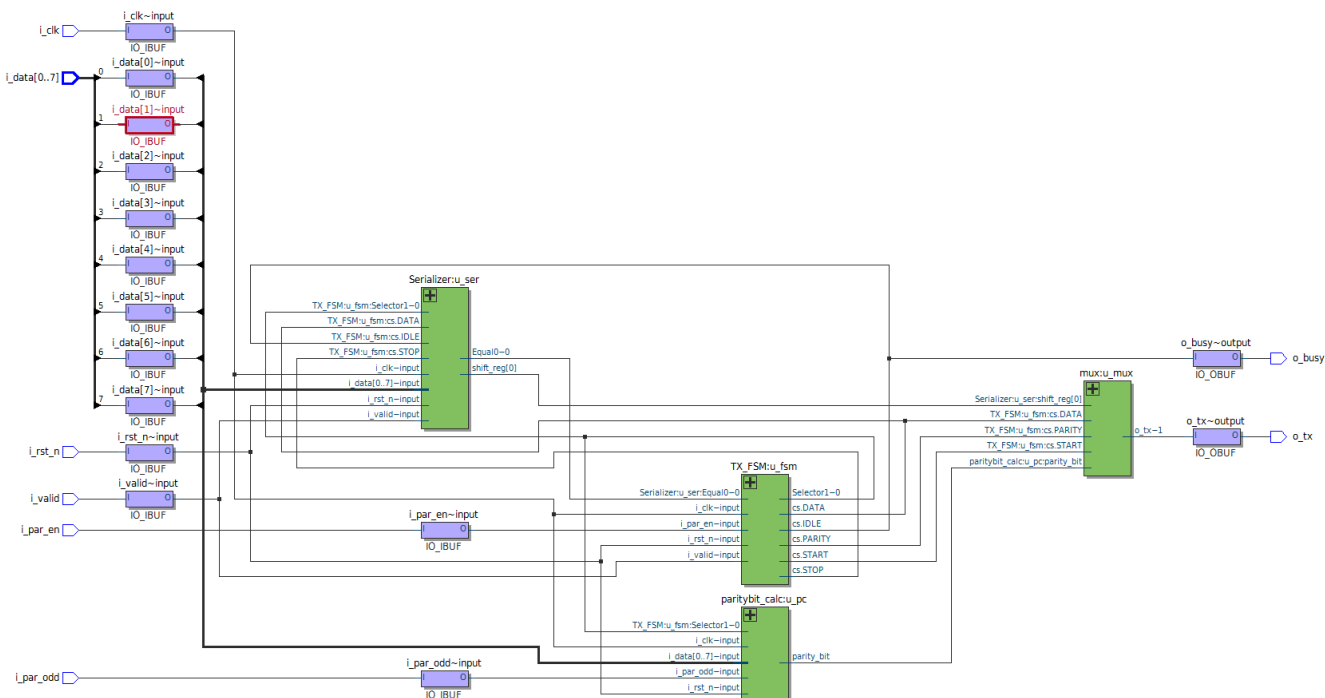
FSM:



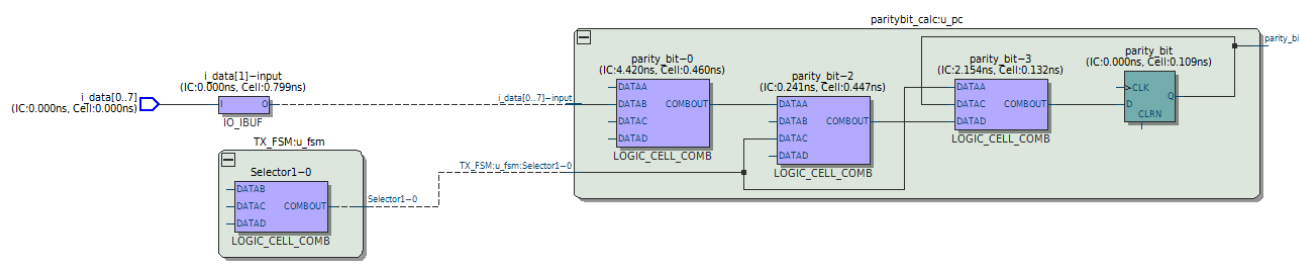
Source State	Destination State	Condition
1 DATA	PARITY	(i_par_en).(ser_done)
2 DATA	DATA	(!ser_done)
3 DATA	STOP	(!i_par_en).(ser_done)
4 IDLE	IDLE	(!i_valid)
5 IDLE	START	(i_valid)
6 PARITY	STOP	
7 START	DATA	
8 STOP	IDLE	(!i_valid)
9 STOP	START	(i_valid)

FPGA Technology Mapping:

(Post_mapping):



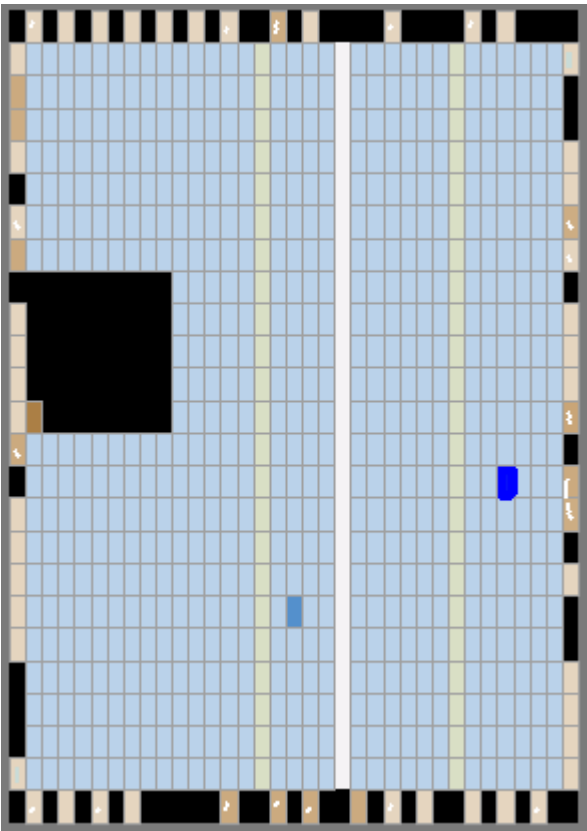
(post_fitting):



Implementation / Resource Utilization:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Sep 28 21:22:14 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	uart_tx
Top-level Entity Name	uart_tx
Family	Cyclone IV E
Device	EP4CE6E22A7
Timing Models	Final
Total logic elements	24 / 6,272 (< 1 %)
Total registers	17
Total pins	15 / 92 (16 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)

Chip_planner:



Timing_Reports:

	Fmax	Restricted Fmax	Clock Name	Note
1	94.11 MHz	94.11 MHz	i_clk	

Setup:

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.374	Serializer:u_ser shift_reg[0]	o_tx	i_clk	i_clk	20.000	-2.382	6.224
2	10.370	i_valid	paritybit_calc:u_pc parity_bit	i_clk	i_clk	20.000	2.289	8.916
3	10.524	i_data[1]	paritybit_calc:u_pc parity_bit	i_clk	i_clk	20.000	2.289	8.762
4	10.545	TX_FSM:u_fsm cs.PARITY	o_tx	i_clk	i_clk	20.000	-2.378	5.057
5	10.550	TX_FSM:u_fsm cs.START	o_tx	i_clk	i_clk	20.000	-2.378	5.052
6	10.697	TX_FSM:u_fsm cs.DATA	o_tx	i_clk	i_clk	20.000	-2.378	4.905
7	10.930	TX_FSM:u_fsm cs.PARITY	o_tx	i_clk	i_clk	20.000	-2.378	4.672
8	10.938	TX_FSM:u_fsm cs.START	o_tx	i_clk	i_clk	20.000	-2.378	4.664
9	10.952	paritybit_calc:u_pc parity_bit	o_tx	i_clk	i_clk	20.000	-2.378	4.650
10	11.080	i_data[3]	paritybit_calc:u_pc parity_bit	i_clk	i_clk	20.000	2.289	8.206

Hold:

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.428	Serializer:u_ser shift_reg[7]	Serializer:u_ser shift_reg[7]	i_clk	i_clk	0.000	0.075	0.692
2	0.429	paritybit_calc:u_pc parity_bit	paritybit_calc:u_pc parity_bit	i_clk	i_clk	0.000	0.074	0.692
3	0.429	TX_FSM:u_fsm cs.IDLE	TX_FSM:u_fsm cs.IDLE	i_clk	i_clk	0.000	0.074	0.692
4	0.429	Serializer:u_ser counter[2]	Serializer:u_ser counter[2]	i_clk	i_clk	0.000	0.074	0.692
5	0.429	TX_FSM:u_fsm cs.DATA	TX_FSM:u_fsm cs.DATA	i_clk	i_clk	0.000	0.074	0.692
6	0.429	Serializer:u_ser counter[1]	Serializer:u_ser counter[1]	i_clk	i_clk	0.000	0.074	0.692
7	0.430	Serializer:u_ser counter[0]	Serializer:u_ser counter[0]	i_clk	i_clk	0.000	0.074	0.693
8	0.443	Serializer:u_ser shift_reg[5]	Serializer:u_ser shift_reg[4]	i_clk	i_clk	0.000	0.075	0.707
9	0.444	Serializer:u_ser shift_reg[1]	Serializer:u_ser shift_reg[0]	i_clk	i_clk	0.000	0.075	0.708
10	0.445	Serializer:u_ser shift_reg[2]	Serializer:u_ser shift_reg[1]	i_clk	i_clk	0.000	0.075	0.709

Constraints_file:

```
# Set time format
set_time_format -unit ns -decimal_places 3

# Define Main Clock (50 MHz = 20 ns period)
create_clock -name {i_clk} -period 20.000 -waveform { 0.000 10.000 } [get_ports {i_clk}]

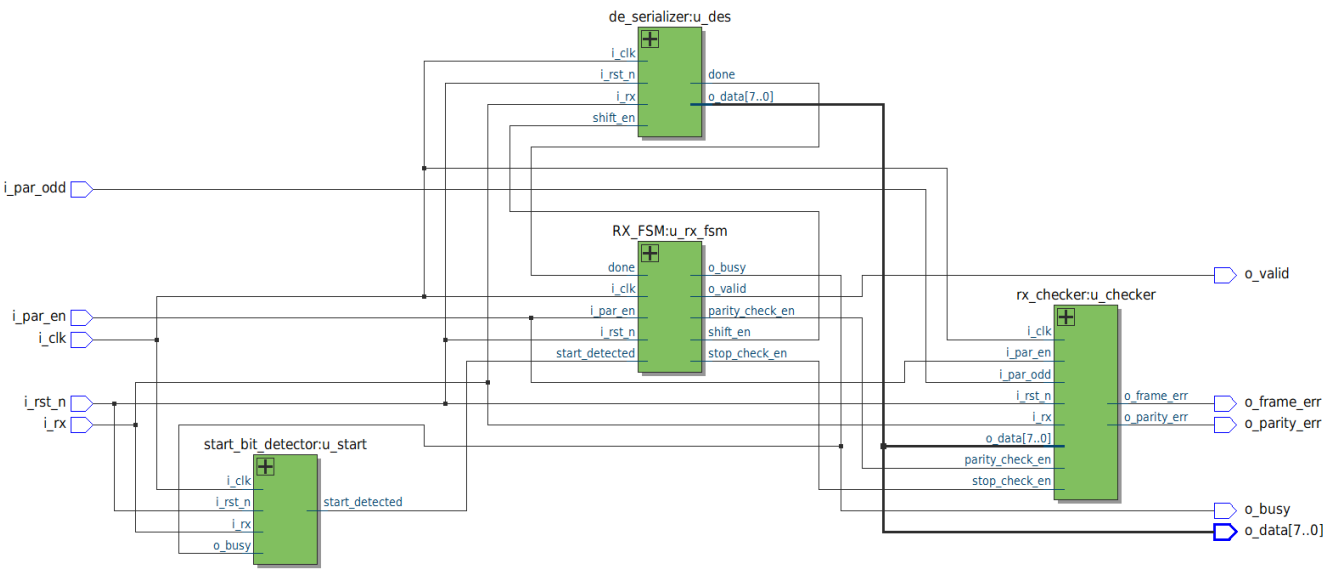
# Automatically derive clock uncertainty
derive_clock_uncertainty

# Input Constraints (Adjust port names to match your design)
set_input_delay -clock [get_clocks {i_clk}] -max 3.000 [all_inputs]
set_input_delay -clock [get_clocks {i_clk}] -min 1.000 [all_inputs]
# Remove clock port from input delay application
remove_input_delay [get_ports {i_clk}]

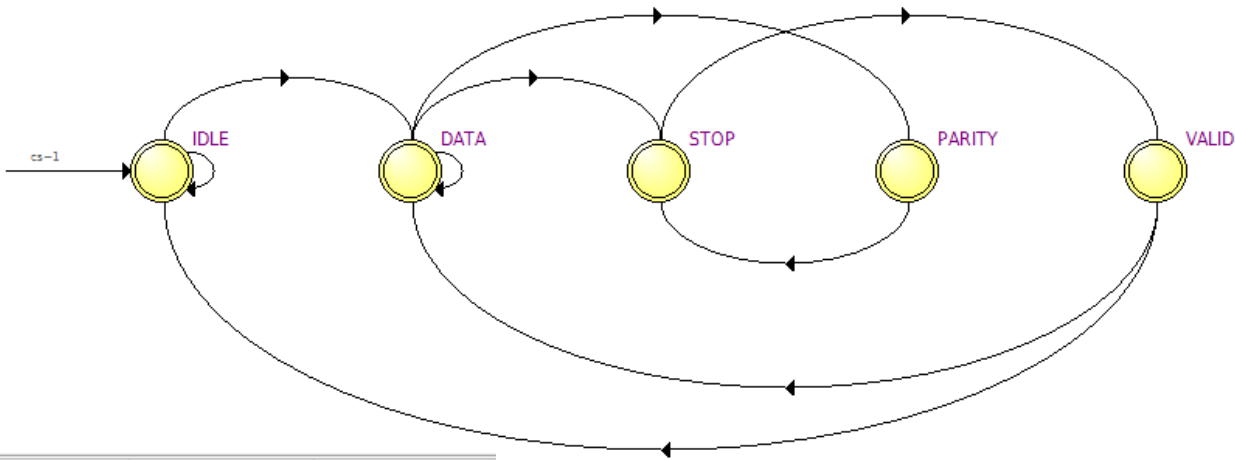
# Output Constraints (Adjust port names to match your design)
set_output_delay -clock [get_clocks {i_clk}] -max 2.000 [all_outputs]
set_output_delay -clock [get_clocks {i_clk}] -min 0.500 [all_outputs]
```

UART RX:

RTL Architecture:



FSM:



Source State	Destination State	Condition
1 DATA	DATA	(!done)
2 DATA	STOP	(!i_par_en).(done)
3 DATA	PARITY	(i_par_en).(done)
4 IDLE	IDLE	(!start_detected)
5 IDLE	DATA	(start_detected)
6 PARITY	STOP	
7 STOP	VALID	
8 VALID	IDLE	(!start_detected)
9 VALID	DATA	(start_detected)

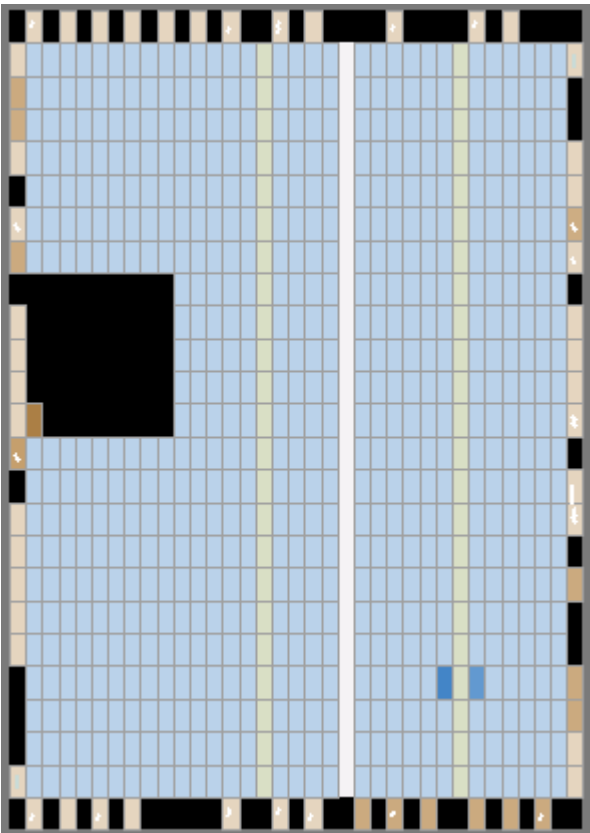
(Post_mapping):



Implementation / Resource Utilization:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Sep 28 23:10:38 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	uart_rx
Top-level Entity Name	uart_rx
Family	Cyclone IV E
Device	EP4CE6E22A7
Timing Models	Final
Total logic elements	24 / 6,272 (< 1 %)
Total registers	18
Total pins	17 / 92 (18 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)

Chip_planner:



Timing_Reports:

	Fmax	Restricted Fmax	Clock Name	Note
1	55.49 MHz	55.49 MHz	i_clk	

Setup:

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.980	i_par_en	o_parity_err	i_clk	i_clk	20.000	0.000	8.000
2	3.530	i_par_odd	o_parity_err	i_clk	i_clk	20.000	0.000	6.450
3	5.524	de_serializer:u_des o_data[3]	o_parity_err	i_clk	i_clk	20.000	-2.377	7.079
4	5.529	de_serializer:u_des o_data[1]	o_parity_err	i_clk	i_clk	20.000	-2.377	7.074
5	5.944	de_serializer:u_des o_data[7]	o_parity_err	i_clk	i_clk	20.000	-2.377	6.659
6	5.957	rx_checker:u_c...red_parity_bit	o_parity_err	i_clk	i_clk	20.000	-2.377	6.646
7	6.274	de_serializer:u_des o_data[2]	o_parity_err	i_clk	i_clk	20.000	-2.377	6.329
8	6.274	de_serializer:u_des o_data[5]	o_parity_err	i_clk	i_clk	20.000	-2.377	6.329
9	6.354	de_serializer:u_des o_data[0]	o_parity_err	i_clk	i_clk	20.000	-2.377	6.249
10	6.371	de_serializer:u_des o_data[4]	o_parity_err	i_clk	i_clk	20.000	-2.377	6.232

Hold:

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.429	de_serializer:u_des bit_cnt[2]	de_serializer:u_des bit_cnt[2]	i_clk	i_clk	0.000	0.074	0.692
2	0.429	de_serializer:u_des bit_cnt[1]	de_serializer:u_des bit_cnt[1]	i_clk	i_clk	0.000	0.074	0.692
3	0.430	de_serializer:u_des bit_cnt[0]	de_serializer:u_des bit_cnt[0]	i_clk	i_clk	0.000	0.074	0.693
4	0.477	de_serializer:u_des o_data[2]	de_serializer:u_des o_data[1]	i_clk	i_clk	0.000	0.074	0.740
5	0.487	RX_FSM:u_rx_fsm cs.VALID	RX_FSM:u_rx_fsm cs.DATA	i_clk	i_clk	0.000	0.074	0.750
6	0.515	RX_FSM:u_rx_fsm cs.DATA	de_serializer:u_des bit_cnt[0]	i_clk	i_clk	0.000	0.074	0.778
7	0.646	de_serializer:u_des o_data[4]	de_serializer:u_des o_data[3]	i_clk	i_clk	0.000	0.074	0.909
8	0.732	RX_FSM:u_rx_fsm cs.DATA	de_serializer:u_des bit_cnt[2]	i_clk	i_clk	0.000	0.074	0.995
9	0.764	RX_FSM:u_rx_fsm cs.PARITY	RX_FSM:u_rx_fsm cs.STOP	i_clk	i_clk	0.000	0.074	1.027
10	0.807	RX_FSM:u_rx_fsm cs.DATA	RX_FSM:u_rx_fsm cs.STOP	i_clk	i_clk	0.000	0.074	1.070

Constraints_file:

```
# =====
# UART RX - Synopsys Design Constraints (SDC)
# =====
# Tool : Intel Quartus Prime
# Device : EP4CE6E22A7 (Cyclone IV E)
# Clock : 50 MHz (Period: 20.000 ns)
# =====

set_time_format -unit ns -decimal_places 3

# -----
# 1. Primary Clock Definition
# -----
create_clock \
    -name {i_clk} \
    -period 20.000 \
    -waveform {0.000 10.000} \
    [get_ports {i_clk}]

# -----
# 2. Clock Uncertainty
# Let Quartus automatically derive jitter and guard-band values
# -----
derive_clock_uncertainty

# -----
# 3. Asynchronous / False Path Signals
# Since UART RX line and Reset are asynchronous to i_clk,
# set false paths to prevent unnecessary timing violations.
# -----
set_false_path -from [get_ports {i_rst_n}]
set_false_path -from [get_ports {i_rx}]

# -----
# 4. Input Timing Constraints (Synchronous Inputs)
# -----
set_input_delay -clock [get_clocks {i_clk}] -max 5.000 [get_ports {i_par_en i_par_odd}]
set_input_delay -clock [get_clocks {i_clk}] -min 1.000 [get_ports {i_par_en i_par_odd}]

# -----
# 5. Output Timing Constraints
# -----
set_output_delay -clock [get_clocks {i_clk}] -max 5.000 [get_ports {o_data[*] o_valid o_busy o_parity_err o_frame_err}]
set_output_delay -clock [get_clocks {i_clk}] -min 1.000 [get_ports {o_data[*] o_valid o_busy o_parity_err o_frame_err}]
```